Journey

Sri Lanka's Bus Booking Platform

Full-Stack PWA · Firebase · PayHere · Google Maps

Kanchana Herath · QA Engineer & AI Enthusiast

> *"A production-grade digital bus ticketing platform for Sri Lanka's inter-city travel market — connecting passengers, bus operators, and administrators in one unified PWA ecosystem."*

PROJECT AT A GLANCE

04	30	12	18
4 Systems	30+ Pages	12 Collections	18 Bugs
Integrated portals	Across all sub-apps	Firestore rules secured	BRD-audited & fixed

PROJECT OVERVIEW

Journey is a production-grade bus booking platform built end-to-end as a solo project, covering the full spectrum of a two-sided marketplace — from passenger-facing seat discovery and booking to operator fleet management, real-time GPS journey tracking, and an administrative control system for financial governance and platform health.

The platform manages the complete booking lifecycle: route search, concurrent-safe seat reservation via Firestore transactions, PayHere payment processing with an escrow model, operator-side journey-day manifest management, post-journey rating and complaint handling, and admin-controlled payment release.

FOUR INTEGRATED SYSTEMS

01 Passenger App

A Windows 11 Fluent-inspired PWA where passengers search routes by date, select seats on an interactive bus layout, pay via PayHere, track their bus live on journey day via Google Maps, and submit post-journey ratings and complaints.

`/ /results /seats /payment /confirmation /profile /track /feedback`

02 Service Provider Portal

A dashboard for bus operators to publish listings, design custom seat layouts, manage schedules and journey dates, view passenger bookings, and declare bus unavailability with automated passenger notifications and refund processing.

`/sp/index /sp/add-bus /sp/buses /sp/bookings`

Journey

Sri Lanka's Bus Booking Platform

Full-Stack PWA · Firebase · PayHere · Google Maps

Kanchana Herath · QA Engineer & AI Enthusiast

03

Admin Control System (ACS)

A full platform governance dashboard covering payment escrow release, refund approval, complaint management with operator rating downgrade, user and bus administration, push notification broadcasting, overlay banner publishing, and a live system configuration panel with 11 feature toggles.

`/admin/dashboard /payments /refunds /complaints /buses /users /config /banners`

04

JourneyBus Operator App

A mobile-first companion app for journey-day operations. Operators load a real-time Firestore passenger manifest, mark onboarded or no-show status, export to CSV/PDF, and declare bus unavailability — triggering automated refund batch processing across all affected bookings in a single transaction.

`/busoperator/ Real-time onSnapshot manifest CSV + PDF export Batch refunds`

TECHNICAL HIGHLIGHTS

Concurrent Seat Booking — Race Condition Fix

Solved the classic double-booking problem without Cloud Functions. A Firestore transaction atomically reads active bookings and seat_lock documents (15-min TTL). If any requested seat is taken, it throws a typed SEAT_CONFLICT error with the conflicting seat IDs. The second passenger is redirected back to seat selection instantly. All in a single atomic write.

Payment Escrow Architecture

Passenger funds sit in paymentStatus:"held" state in Firestore after payment. They are released to operators only after the journey date passes and admin approval. If a critical complaint is open, that payment stays in escrow until resolved. Every state change is written to an immutable audit_logs collection.

Real-Time Passenger Manifest

Firestore onSnapshot listeners drive a live-updating manifest on journey day. When the operator taps "Onboarded", the row turns green across all connected devices in under a second. No polling, no refresh, no server round-trips — pure Firestore real-time sync.

PWA with Smart Offline Resilience

The Service Worker precaches static assets for offline use, but deliberately excludes config.js from the cache. This prevents stale API keys, feature flags, and Firebase config from being served to users — a common PWA security oversight.

Role-Based Firestore Security Rules

Database-layer authorization covers 12 collections. Passengers only read their own bookings. Operators update only onboard fields, never payment data. The fields total, seats, and paymentStatus are immutable after booking creation. paymentStatus changes are admin-only.

Bilingual i18n — EN + Sinhala

Full English / Sinhala bilingual support via a custom i18n module with en.json and si.json dictionaries. Applied to dynamically generated HTML and static content alike. Language preference persists via localStorage with English fallback.

TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	HTML5 · CSS3 · Vanilla JS ES2020+	Multi-portal SPA, zero framework dependency
PWA	Service Worker · Web Manifest	Offline resilience, install-to-homescreen
Database	Firebase Firestore (real-time)	Seat locks · escrow state · live manifest
Auth	Firebase Authentication	Email/password · role-based access control
Payments	PayHere (SL gateway)	Escrow model · per-call handler scoping
Email	EmailJS	Booking confirmations · cancellation alerts
Maps	Google Maps JavaScript API	Real-time GPS tracking on journey day
Hosting	Firebase Hosting	All 4 portals from one deployment
Design	Windows 11 Fluent · Phosphor Icons	DM Sans + Sora · CSS custom properties

i18n	Custom module · en.json · si.json	English + Sinhala bilingual support

SCALE & SCOPE

4 fully integrated portals	Passenger, Service Provider, Admin Control System, JourneyBus Operator App
30+ HTML pages	Across all four sub-applications
18 BRD bugs audited and fixed	Plus 1 critical concurrent-booking race condition fix
11 Firestore composite indexes	Supporting complex filtered multi-field queries
12 Firestore collections secured	Field-level write restrictions on all payment-critical fields
45+ official SL bus routes	With intermediate stop and city autocomplete support
Bilingual (EN + Sinhala)	All passenger-facing surfaces translated

WHAT THIS DEMONSTRATES

Marketplace Architecture — Designed a two-sided marketplace from BRD requirements through production-ready code, balancing passenger, operator, and admin workflows in a single coherent product.

Security-First Database Design — Zero client-side trust. Authorization enforced at the Firestore rules layer with role checks, ownership validation, and field-level immutability on all financial data.

Concurrency Problem Solving — Resolved a real-world race condition using Firestore transactions and TTL-based seat locks at the application layer — without Cloud Functions or any server infrastructure.

Full-Stack Solo Ownership — Single-handedly owned UX design, frontend engineering, Firestore schema, security rules, payment integration, i18n, PWA configuration, and deployment pipeline.

Product & Business Thinking — Designed escrow flow, tiered refund policy, complaint severity classification, operator rating downgrade logic, and admin feature toggles as coherent business rules — not just UI.

RESUME / LINKEDIN ONE-LINER

> *Built **Journey**, a full-stack bus booking PWA for Sri Lanka with concurrent-safe seat reservation via Firestore transactions, a PayHere escrow payment model, four integrated portals (passenger, operator, admin, journey-day manifest app), real-time onSnapshot passenger manifest, and a complete role-based Firestore security rule architecture covering 12 collections with field-level write restrictions.*

Firebase · Firestore · Vanilla JS · PWA · PayHere · Google Maps · EmailJS · Fluent Design · i18n · Sri Lanka

Kanchana Herath — QA Engineer & AI Enthusiast

kanchanaherath251@gmail.com · +94 72 543 4820 · linkedin.com/in/kanchana-herath-9b831a339